CSE113: Parallel Programming

Homework 1 Report

Instruction-Level Parallelism and C++ Threads

Brian Lee
blee134@ucsc.edu

October 5, 2026

Contents

Contents 2

Part 1 3

Design . . . 3

Results . . . 3

Analysis . . . 4

Part 2 5

Design . . . 5

Results . . . 5

Analysis . . . 5

Part 3 7

Design . . . 7

Results . . . 7

Analysis . . . 8

Experimental Environment and Limitations 8

Part 1

Design

The two Python functions generate C++ implementations of the supplied element-wise floating-point loop. Both accept a dependency-chain length and unroll factor and return a complete C++ function. The generated loop advances by the unroll factor; because that factor divides the array size, no cleanup loop is required.

Sequential generator. For each unrolled element, it loads one temporary, emits every addition in that element's chain, stores the result, and only then begins the next element. This saves loop-control work but intentionally keeps dependency chains serialized.

Interleaved generator. It loads all temporaries first, emits addition *j* for every temporary before addition *j+1*, then stores all results. Each element receives the same ordered floating-point operations as the reference, while independent chains become available together. All constants use the required single-precision `f` suffix.

Results

Chain length was 64; the array contained 8,388,608 floats. Each point is the arithmetic mean of five runs compiled with `-std=c++14 -fno-unroll-loops -O0`.

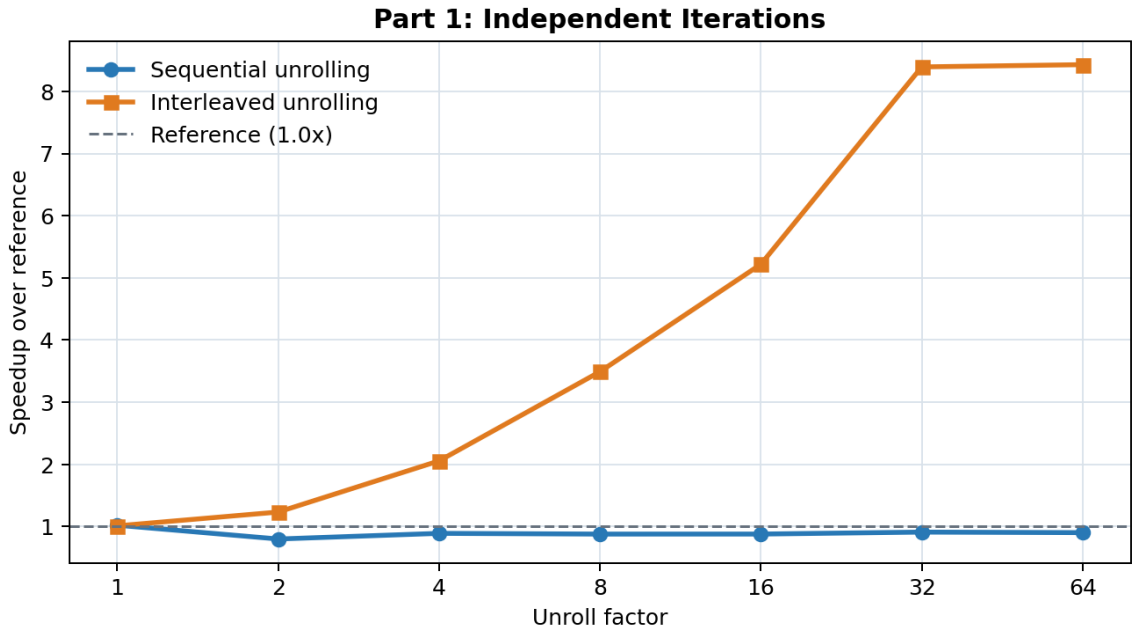


Figure 1. Part 1 speedup relative to the reference implementation.

Table 1. Part 1 mean timing results; five runs per point.

Unroll	Reference (s)	Sequential (s)	Interleaved (s)	Seq. speedup	Int. speedup
1	0.8543	0.8376	0.8462	1.020	1.010
2	0.8649	1.0845	0.7026	0.798	1.232
4	0.9162	1.0306	0.4469	0.889	2.055
8	0.8609	0.9811	0.2465	0.877	3.491
16	0.8385	0.9554	0.1607	0.878	5.219
32	0.8587	0.9444	0.1023	0.909	8.394
64	0.8476	0.9412	0.1005	0.901	8.430

Correctness: an independent harness compared both generated loops element-by-element with the reference for five chain lengths and seven unroll factors. All 35 combinations passed locally and in the course Docker image.

Analysis

The sequentially unrolled loop remains near the reference and is slower for every factor above one (about 0.80-0.91x). Completing one chain before another exposes no extra ILP; with optimization disabled, additional temporaries and code size can outweigh saved branches.

Interleaving shows the expected ILP trend: speedup rises from 1.01x at factor 1 to 3.49x at factor 8 and 8.39x at factor 32. Independent chains fill pipeline slots that would otherwise wait on one chain's data dependency. The curve is nearly flat from 32 to 64 (8.39x versus 8.43x), indicating saturation of execution width and latency hiding; further unrolling mostly adds code size and register pressure.

Part 2

Design

The reduction generator replaces one loop-carried accumulator with *P* independent partial sums. Array elements 0 through *P*-1 hold those sums. The main loop starts at index *P*, advances by *P*, and updates each partition once. A cleanup loop combines accumulators 1 through *P*-1 into element 0. Power-of-two input size and partition count guarantee complete coverage without a remainder loop.

Results

Five runs were measured for partition counts 1, 2, 4, 8, and 16. The timing copy used 134,217,728 doubles per array (2 GiB total), a permitted power-of-two reduction from the 8 GiB default allocation; submitted constants were not changed. Tests used the actual checked-in `-O0` command, although its nearby comment mentions `-O3`.

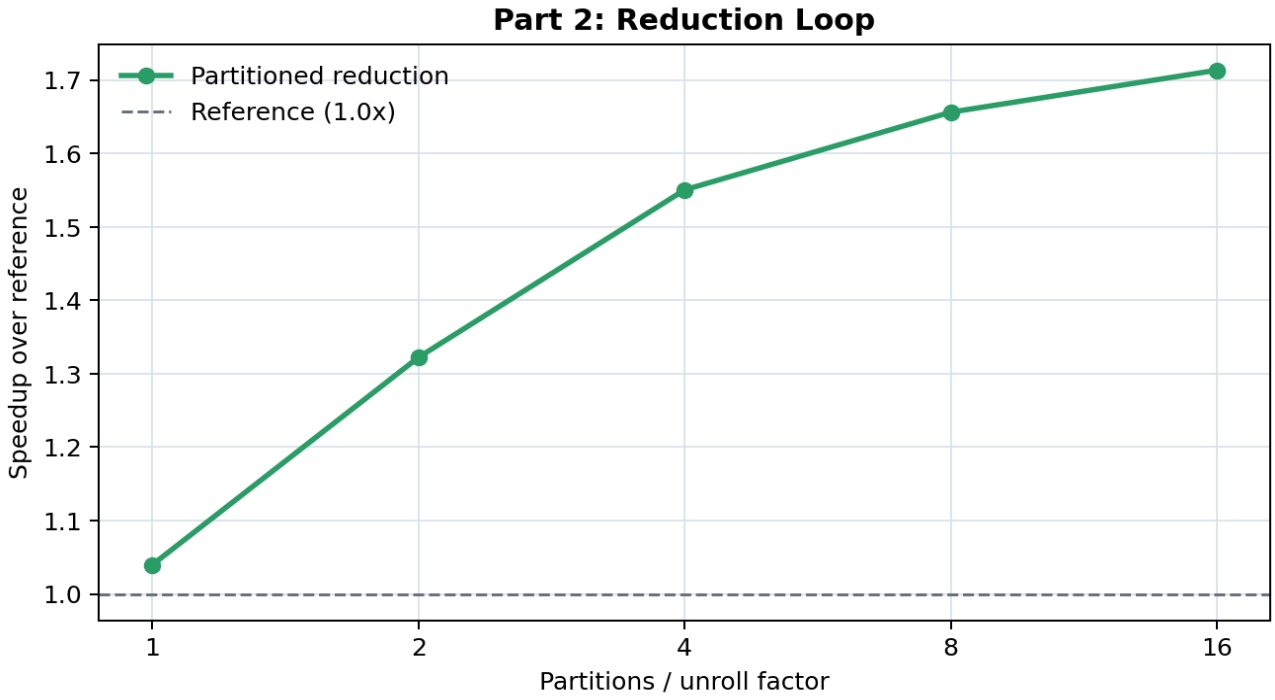


Figure 2. Part 2 reduction speedup relative to the reference.

Partitions	Reference (s)	Partitioned (s)	Speedup
1	0.1591	0.1528	1.040
2	0.1317	0.0999	1.322
4	0.1918	0.1237	1.550
8	0.1246	0.0752	1.656
16	0.1393	0.0813	1.713

Table 2. Part 2 mean timing results; five runs per point.

Correctness: arrays of 1,024 ones were tested with partition counts 1 through 32; every case produced exactly 1,024 in element 0. The same suite passed in the course Docker image.

Analysis

Speedup increases monotonically from 1.04x with one accumulator to 1.71x with sixteen. The reference carries one dependency through every addition. Partial sums split it into independent chains, allowing overlap. Gains taper after eight partitions because addition latency is substantially covered and memory bandwidth, loop instructions, cleanup work, and finite execution resources dominate. Absolute times varied more than ratios, consistent with a short memory-intensive benchmark sharing the system with ordinary OS activity.

Part 3

Design

The sequential function visits each location in order and performs all *K* required volatile increments before moving on. Round-robin worker *tid* starts at index *tid* and advances by the thread count, exactly implementing *i* mod *N* = *tid*. All workers are joined before return.

The custom implementation uses contiguous blocks. Worker *tid* receives [*tid* x size / *N*, (*tid*+1) x size / *N*). Integer endpoints cover the full array even when division is uneven. Each worker directly increments volatile memory and finishes one location before advancing. Contiguous ownership minimizes cache-line sharing.

Results

The experiment used SIZE=1024 and K=1,048,576. Sequential execution averaged about 0.75 seconds, close to the requested one-second target. Each point is the mean of five runs compiled with `-O0`.

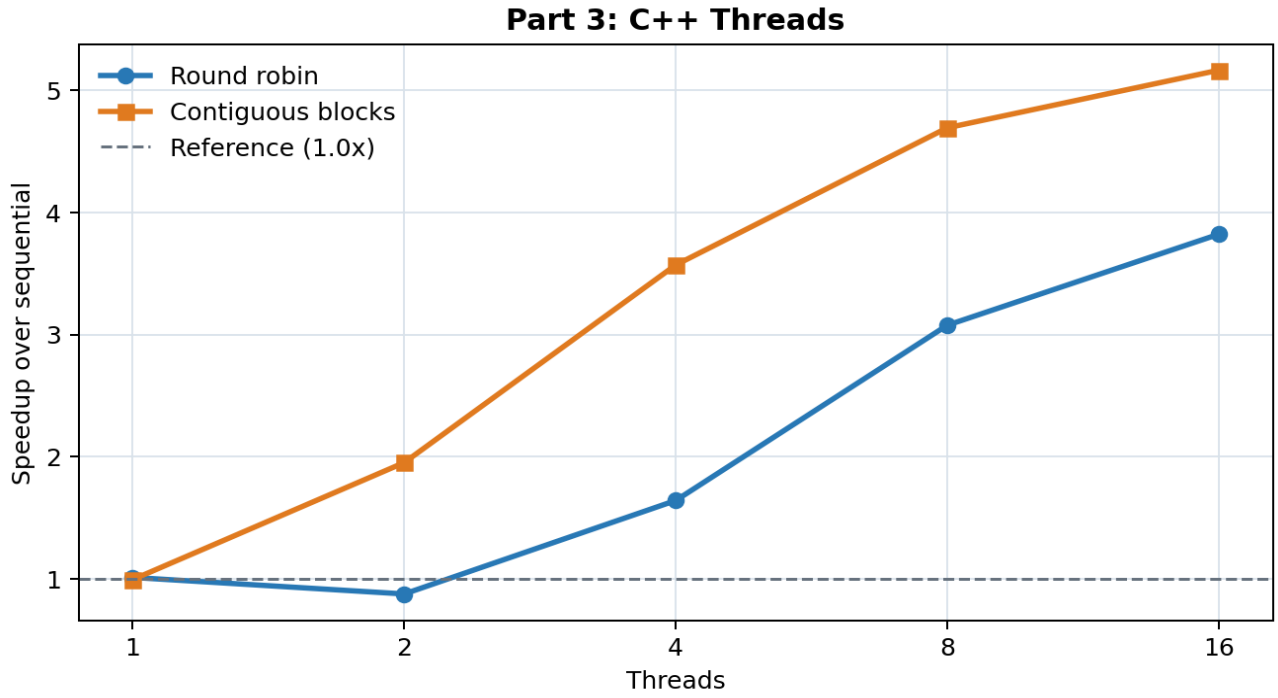


Figure 3. Part 3 speedup relative to sequential execution.

Threads	Sequential (s)	Round robin (s)	Contiguous (s)	RR speedup	Custom speedup
1	0.7438	0.7346	0.7475	1.013	0.995
2	0.7450	0.8579	0.3817	0.877	1.953
4	0.7549	0.4598	0.2116	1.642	3.569
8	0.7528	0.2466	0.1605	3.076	4.692
16	0.7460	0.1954	0.1446	3.821	5.163

Table 3. Part 3 mean timing results; five runs per point.

Correctness: 1,031 elements and K=257 were tested with 1, 2, 4, 8, and 16 threads, intentionally exercising uneven block division. Every element of all three arrays equaled K locally and in the course Docker image.

Analysis

Contiguous blocks scale much better than round robin. At two threads they reach 1.95x while round robin falls to 0.88x. Adjacent round-robin elements share cache lines, so different cores repeatedly invalidate one another's lines despite updating different integers. This false sharing is especially costly for volatile read-modify-write operations. Contiguous blocks give nearly every line a single owner; at two and four threads this strategy is about 2.25x and 2.17x faster than round robin.

Both curves flatten. Custom speedup reaches 4.69x at eight threads and 5.16x at sixteen; round robin reaches 3.82x. Sixteen software threads oversubscribe the machine's ten heterogeneous cores. Thread startup/join costs, memory traffic, and unavoidable volatile operations also make scaling sublinear.

Experimental Environment and Limitations

Measurements were collected on an Apple M4 MacBook Air with 10 CPU cores and 16 GiB memory, using Apple clang 17.0.0 on macOS. Correctness was also verified with g++ in the supplied reeselevine/cse113:latest course image. Timings were collected natively to avoid macOS container-virtualization effects, so trends are meaningful but exact values are platform-dependent. Threads were not pinned, dynamic frequency scaling remained enabled, and results are arithmetic means rather than confidence intervals.

> No homework implementation files were changed during testing; experiments ran from temporary generated copies.